When Is It Too Late to Squeeze? Remedy-Specific Recovery Deadlines under Observation Age and Response Delay

Nila Karthikesan
Independent research
https://github.com/nilakarthikesan/grasp-recovery-under-delayed-feedback

Abstract—A runtime monitor can detect an impending grasp failure accurately and still be useless if, by the time its command takes effect, no available remedy can change the outcome. We measure how the value of three bounded recovery actions—slowing, squeezing to the actuator bound, and placing down to regrasp—depends jointly on the age of the observation the supervisor saw, the delay before its command acts, and hidden mass, friction and an exogenous push. In a MuJoCo grasp–transport–place task with a force-limited parallel-jaw gripper, we save the complete system at every control step after acquisition and run every remedy after every delay from the same state: 7,394 decision states, 88,728 matched branches, bit-exact replay. With one intervention per episode, any supervisor's closed-loop outcome is an exact table lookup, verified against live re-simulation. We find remedy-specific deadlines: a 50% recovery chance needs 50 ms of lead for squeeze but 350 ms for place/regrasp, both shifting with response delay; slowing recovers at most 22% of lost states. Every nominal loss in the test set stayed recoverable by some remedy even at 500 ms delay, so within this task the deployable gap is detection under stale evidence, not physics. A reactive slip detector loses 28 completion points across the timing grid; anticipatory selectors lose 11. A value model conditioned on its own latency changes what and how often it intervenes but does not complete more tasks than a future-loss predictor driving a fixed squeeze (-1.6 points, 95% episode-clustered CI $[-3.5, -0.2]$). Code, protocol, per-episode outcomes and controls are released.

I. Introduction

Failure detectors for manipulation policies are well studied [6], [7], and recent work evaluates whether an alarm can still be acted on by branching counterfactual interventions from restored states [1], [2], [3]. Such evaluations usually fold together the age of the evidence the monitor saw and the delay before its command takes effect, and each remedy has its own deadline relative to both: a detector that is accurate on 250 ms-old evidence is irrelevant if the only remedy that still works needed the command 300 ms earlier.

We ask a narrow, measurable question: how do observation age and response delay, separately, change the value of different recovery actions under hidden contact dynamics, and does a supervisor that knows its own latency choose better? We answer it in simulation with a protocol in which the counterfactual that defines "value" is exact. Contributions:

1) A grasp–transport–place task with hidden mass, friction, grasp offset and push, a frozen controller, a bounded remedy menu on deployable inputs, and bit-exact snapshots enabling matched remedy $\times$ delay branches from every decision state.
2) Recovery frontiers per remedy versus decision lead and response delay, showing remedy-specific deadlines (Fig. 1).
3) Six supervisors evaluated on a 3×4 age $\times$ delay grid under a validation intervention budget, with episode-clustered bootstrap intervals, feedback-null controls and a held-out friction band (Fig. 2, Table I).
4) A negative result with intervals: latency-conditioning the intervention-value model did not improve completion over anticipatory detection with a fixed squeeze.

II. Relation to prior work

The distinction between detecting a failure and being able to act on it is established. RESCUE [1] restores simulator and policy state at checkpoints, branches counterfactual interventions, and shows that recoverability depends on the state–intervention pair, with a recoverability horizon conditional on the intervention repertoire; CheckVLA [2] defines an intervention-relative irreversibility time against a fixed repair set; PREFAIL [3] labels the latest actionable intervention time in lift-and-place. Kintsugi-VLA [4] and RoboRecover [5] measure recoverability from restored or replayed intermediate states. We do not claim to originate counterfactual branching or recovery deadlines. Our narrower addition is to separate observation age from response delay as explicit axes, to measure how each remedy's deadline shifts with delay from bit-exact matched states in a contact-rich grasp task with hidden physics, and to test a learned selector that conditions on its own latency, with supervisors restricted to deployable sensing. FAIL-Detect [6] and SAFE [7] provide the detection comparators; real-time chunking [8] and ReflexBench [9] address latency for the policy, whereas here the policy is fixed and the supervisor is late. Slip-prediction work [10] motivates the menu; squeezing and slowing are established remedies.

III. Protocol

A. Task, hidden physics and observations

A Cartesian carrier (x, z prismatic joints, stiff gravity-compensated position servos) holds a parallel-jaw gripper whose fingers are force-controlled motors bounded at 7 N. A 4 cm cube rests on a table; the controller approaches, closes at 4 N, lifts 10 cm, transports 0.45 m, lowers until a wrist-load change indicates support contact, releases, retreats and dwells 1 s. Completion (placed) requires the cube within 3 cm of the target in each horizontal coordinate, tilt $< 15°$, speed < 2 cm/s after the dwell. Other labels: lost (left the hand before an intended release), misplaced, preserved abort, acquisition failed, timeout.

Hidden per episode: mass $\sim U(0.15, 0.50)$ kg, sliding friction $\sim U(0.35, 0.80)$ (the object geom dominates every contact; the effective coefficient is read back from the contact array), grasp offset $\sim U(-8, 8)$ mm, and a push: a force on the object equal to its mass times $a_p \sim U(0, 8)$ m/s^2 along $(0.6, 0, -0.8)$ for 200 ms, starting 1–10 control steps into transport. The nominal plan, known to the controller, samples speed $\{0.8, 1.2\}$ m/s and acceleration $\{5, 8, 12\}$ m/s^2. The grasp line sits 3 cm above the cube's centre, so lateral acceleration produces a pendulum torque; among the 100 nominal losses, the interval from a 3 mm in-hand descent to loss is 50–950 ms (median 350 ms), which creates a recovery window.

Deployable observations (20 values at 20 Hz): carrier and finger positions/velocities, finger actuator force, wrist force/torque, last command. Supervisor features: a 4-step window ending at the observed time plus finger gap, steps since acquisition, phase, planned speed/acceleration, and wrist-load and gap change since lift (87 features). Object pose, in-hand descent, tilt, contacts, mass, friction and push are privileged: labels and oracles only.

B. Timing contract

For a decision at control step t: the observation was captured at $t-a$ (observation age), the remedy is requested at t and takes effect at $t + d$ (response delay). Physics never pauses; between t and $t + d$ the controller keeps executing its committed plan. Grid: $a \in \{0, 100, 250\}$ ms, $d \in \{0, 100, 250, 500\}$ ms. Age changes only what the supervisor sees and delay only the physics, so branch outcomes are indexed by (state, remedy, d) and features by (state, a).

C. Matched branches and exact closed-loop evaluation

At every control step in the lift and transport phases of an acquired episode we save MuJoCo's integration state (incl. warm-start), the latched observation and the controller's state; restoring and continuing reproduces the parent bitwise (tested). From each state we execute slow (remaining speed and acceleration $\times 0.4$), squeeze (finger force 4 $\rightarrow$ 7 N) and place/regrasp (decelerate, lower to support contact, open, re-close at 7 N, re-lift, resume at half speed; aborts if the re-close captures nothing) after each delay. Continuation is the parent's own future. 400 episodes (seed 0) gave 314 acquired, of which 100 (31.8%) were lost nominally; 7,394 decision states and 88,728 branches in 5.6 min on 9 CPU cores.

A supervisor cannot change the world before it fires, and we allow one intervention per episode, so its closed-loop outcome at (a, d) is exactly the branch outcome of the first state at which it fires. We verified this on 60 random test episodes and timing cells by re-simulating the learned supervisor live from its own observation history: 60/60 identical outcomes, firing steps and requested remedies.

D. Supervisors

All learned components are gradient-boosted trees (scikit-learn) fitted on training episodes; each supervisor's threshold is chosen on validation to maximise pooled completion subject to intervening in at most 40% of episodes (the nominal loss prevalence is 32%), then frozen. Nominal never intervenes. Random intervenes at the delay-aware selector's realised rate, at a random eligible step with a random remedy (10 seeds). Reactive slip detector + squeeze predicts the privileged current slip label at the observed time and fires squeeze. Future-loss predictor + squeeze predicts loss under continuation and fires squeeze. Value selector, delay-agnostic: per-remedy completion models $\hat{p}_r(x)$ on stale features only, trained on the full timing mixture; fires $\arg\max_r[\hat{p}_r - \hat{p}_{\text{cont}}]$ when that value exceeds τ. Value selector, delay-aware: identical, plus $(a, d, a{+}d)$ as inputs—the latency-conditioned intervention-value model. Privileged menu oracle: in nominally lost episodes, the earliest state and remedy whose branch completes at this delay. Splits are by parent episode (157/63/94 acquired episodes); all ages, decisions and branches of an episode stay together. Uncertainty is a paired cluster bootstrap (2,000 resamples) whose resampling unit is the parent episode: the 12 timing cells of an episode are drawn together, since they share the same physics and are not independent replications. Intervals are therefore about $\sqrt{12}$ wider than a row bootstrap over episode–cell pairs would give. Per-episode, per-cell outcomes for every supervisor are released so the intervals can be recomputed.

IV. Results

A. Remedy-specific deadlines

Fig. 1 shows recoverability from privileged branch labels. We define a remedy's deadline as the smallest lead bin (with at least 20 states) in which at least half the branches complete; it is a summary of the curve, not a boundary beyond which recovery is impossible. squeeze needs a lead of 50/150/300/550 ms at delays 0/100/250/500 ms, and place/regrasp 350/450/600/850 ms. The 300 ms separation at every delay is consistent with the duration of the regrasp sequence, during which the object keeps slipping, though we have not isolated that cause. slow

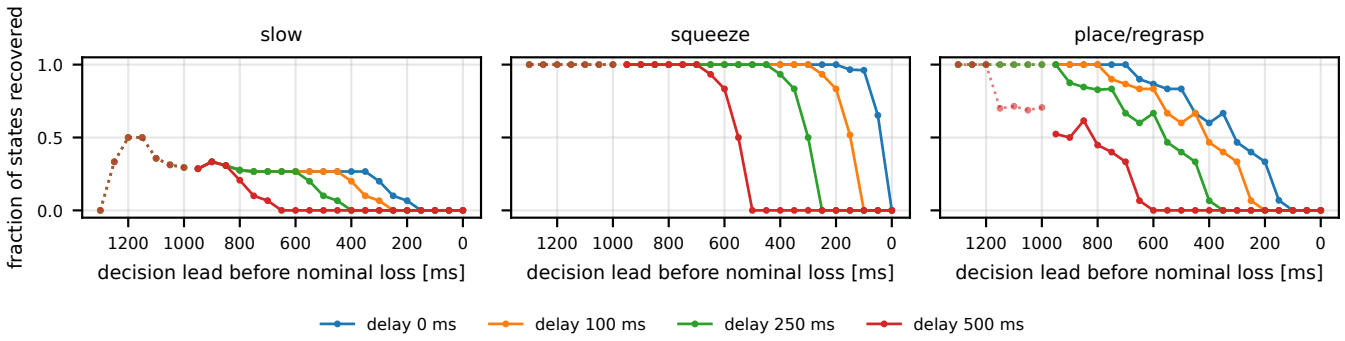


Fig. 1. Recovery frontiers (test episodes that nominal continuation loses; 637 decision states): fraction of states from which each remedy completes the task, versus decision lead before nominal loss, per response delay. Points are empirical bin averages, not calibrated probabilities; dotted segments are bins with fewer than 20 states. Deadlines are remedy-specific and shift by roughly the delay. slow recovers 22% of lost states pooled over leads (9% at 500 ms delay) because most losses are triggered by the push, not by transport acceleration. place/regrasp is non-monotone (recovery islands), so a single "time-to-failure" deadline would misdescribe it.

TABLE I
Pooled over 12 cells × 94 test episodes. Gain: paired difference vs. nominal, 95% episode-clustered CI. Rate: episode–cells with an intervention. Rescue: nominal losses converted. Random: mean of 10 seeds.

Supervisor	Compl.	Gain [95% CI]	Rate	Rescue
Nominal	.681	–	.00	.00
Random @ budget	.739	+.058 [.039, .077]	.42	.19
Reactive slip + squeeze	.852	+.171 [.109, .234]	.43	.54
Future loss + squeeze	.946	+.265 [.178, .348]	.48	.83
Value, delay-agnostic	.934	+.253 [.165, .334]	.45	.79
Value, delay-aware	.930	+.249 [.162, .329]	.42	.78
Oracle (privileged)	1.000	+.319 [.223, .415]	.32	1.00

has no deadline by this definition: pooled over lost states it recovers 22/19/16/9%, reaching 50% only in two 10-state bins at 1.15–1.2 s of lead; 77 of the 100 nominal losses begin at or after the push, which slowing cannot counter. The oracle completes 100% of test episodes at every delay by intervening in the 30 nominally lost ones (31.9%; median lead 1.05 s), so within this task and menu every nominal loss was recoverable; what separates deployable supervisors from the oracle is knowing when and what to fire from stale, non-privileged evidence.

B. Supervisors across the timing grid

Fig. 2 and Table I give the main comparison. Every supervisor improves on nominal; random intervention at the delay-aware selector's realised rate gains only 5.8 points, so intervention frequency does not explain the gains. The reactive slip detector is perfect with fresh evidence and no delay (1.00) but falls to 0.72 at 250 ms age and 500 ms delay: 28 points across the grid. Anticipatory supervisors span 11 points (future-loss 0.99→0.88; delay-aware value 0.98→0.87) because they fire with a median lead of 0.8–0.9 s versus 0.35 s, inside the squeeze deadline even at 500 ms delay. For every supervisor the full 0–500 ms delay sweep costs more completion than the full 0–250 ms age sweep (the ranges differ, so this is not a per-millisecond comparison).

C. Does latency-conditioning help?

It changes behaviour. The delay-aware selector's intervention rate, averaged over ages, falls monotonically with delay (0.472→0.379) while the agnostic selector's is constant by construction, and with 250 ms-old evidence it shifts toward place/regrasp (22 of 44 interventions at zero delay, 17 of 36 at 500 ms, versus 13 of 43 for the agnostic model): the slower, more decisive remedy when evidence is stale. It does not complete more tasks: pooled paired differences are −0.4 points versus the agnostic selector (CI $[-1.8, +0.8]$) and −1.6 points versus future-loss + squeeze (CI $[-3.5, -0.2]$). Giving every learned supervisor a latency-tuned operating point (one validation threshold per cell) does not change this: −0.6 [−1.9, +0.4] and −0.4 [−1.9, +1.2]. The 40% cap is applied on validation; realised test rates were 0.43–0.48 for the learned supervisors, which we report rather than re-tune. Fig. 3 shows the full budget–completion curves: the three anticipatory supervisors stay within 3.5 points of each other up to 60% intervention, and none reaches the oracle below 99% intervention, i.e. squeeze everything, the right-hand end of every curve.

D. Feedback-null controls

With the delay-aware selector frozen, we replaced its observation features at test time, keeping timing features and labels. Frozen observations (first decision state repeated) drop pooled completion from 0.930 to 0.810 (−12.0 points, CI $[-17.8, -6.6]$); observations swapped from another episode at the same step give 0.838 (−9.2, $[-16.8, -1.4]$). Training-mean masking gives 0.894 (−3.6, $[-7.6, +0.7]$) but degenerately: the decision then depends only on timing inputs, and the selector squeezes every episode in eight cells and none in four (67%), so it is not

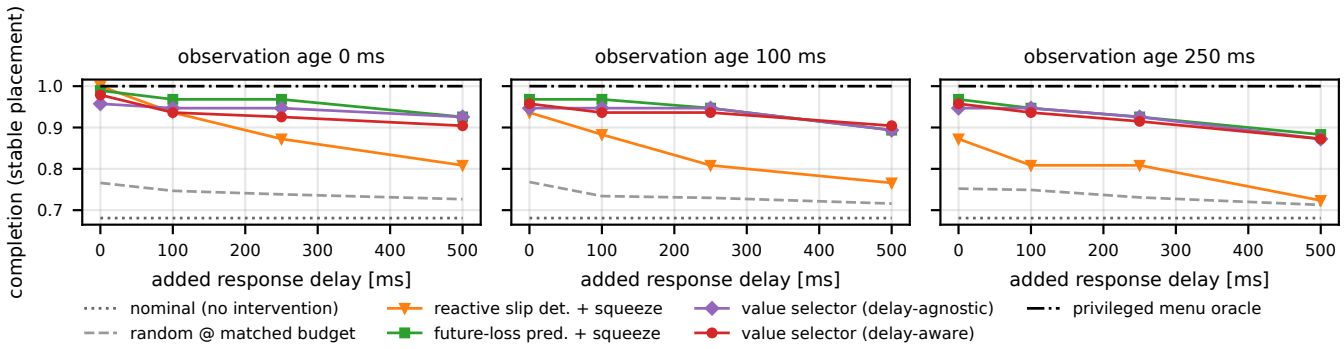


Fig. 2. Closed-loop completion, 94 held-out episodes per cell (cells share episodes). The reactive detector degrades steeply with age and delay; anticipatory supervisors degrade gently and track each other.

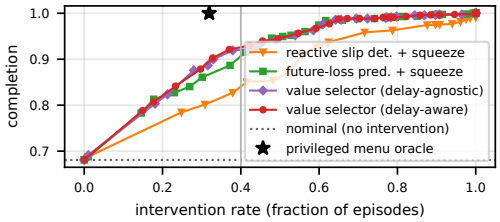


Fig. 3. Completion versus intervention rate (test, pooled) as each threshold is swept. Vertical line: validation budget; star: oracle.

an equal-budget comparator. The selector's gain depends on informative, episode-specific observations.

E. Held-out friction band

Re-splitting so that friction $\in [0.50, 0.58]$ (64 acquired episodes) is never seen in training or validation: nominal 0.5625; future-loss + squeeze 0.814; delay-agnostic 0.829; delay-aware 0.8125; oracle 1.000 at 44% intervention. The delay-aware model again trails: -1.7 points versus agnostic (CI $[-3.8, 0.0]$) and -0.1 versus future-loss ($[-1.2, +0.8]$); here the agnostic selector edges out future-loss, so the ordering between those two is not stable. Completion is lower than on the random split because this band is where the nominal grasp is most often marginal.

V. Discussion and limits

Inference costs 3–12 ms per decision (Apple M4, scikit-learn), inside one control step. (i) Within this task and menu, recoverability is not the bottleneck: the oracle completes every test episode at 500 ms delay. The bottleneck is the evidence race—whether non-privileged observations identify the episode before the remedy's deadline minus the delay. (ii) Because squeeze is the fastest remedy and, within its deadline, almost always sufficient, the value of the richer menu collapses to "squeeze early enough", which a future-loss predictor already does. The delay-aware model learns the right qualitative behaviour without a completion benefit; with 94 clustered test episodes its difference intervals span about ± 1.5 points, so a small benefit is not excluded, but one comparable to the reactive-vs-anticipatory gap is. (iii) Reactive detection, which many systems deploy, is the one design that latency breaks.

The claims are bounded by: simulation only, soft contacts standing in for real friction; one rigid cube; an ideal Cartesian carrier; scripted controller and remedies; deterministic physics with a single push, so paired outcomes are binary, not calibrated probabilities; synthetic fixed delays, not measured latency traces; proprioception and wrist force only; 94 test episodes; a validation-set budget rather than matched realised budgets; and a menu in which squeezing is cheap—objects with a crush limit, or a gripper already at its force bound, would change the frontier and could restore the value of a menu-aware selector. Those are the next experiments, then matched (not counterfactual-exact) blocks on a physical arm. Code, the report with every number above, per-episode outcomes, lookup-verification records and this manuscript are in the repository in the title block.

References

[1] D. Ahmedt-Aristizabal, A. Khamis, M. A. Armin, L. Petersson, "RESCUE: Is this alarm worth acting on? Counterfactual evaluation of VLA runtime monitors," arXiv:2610.13516, 2026.
[2] Y. Liu et al., "CheckVLA: Execution-time verification with action-conditioned world model," arXiv:2607.26789, 2026.
[3] Z. Shangguan, R. Chitale, R. Patel, S. K. Gupta, D. Seita, "PREFAIL: Identifying precursors to failures in robotic lift-and-place tasks," arXiv:2607.16921, 2026.
[4] I. Snegirev et al., "Kintsugi-VLA: Turning failed robot rollouts into recovery data through interventional recoverability," arXiv:2609.31048, 2026.
[5] Y. Li et al., "RoboRecover: Benchmarking robot policy recovery under execution deviations," arXiv:2609.28952, 2026.
[6] C. Xu et al., "Can we detect failures without failure data?" RSS, 2025.
[7] Q. Gu et al., "SAFE: Multitask failure detection for vision-language-action models," NeurIPS, 2025.
[8] K. Black, M. Y. Galliker, S. Levine, "Real-time execution of action chunking flow policies," NeurIPS, 2025.
[9] Y. Chen, W. Zhang, X. Li, "Reflex: Fast and predictive VLA models for reaction-critical manipulation," arXiv:2608.14379, 2026.
[10] F. Veiga, B. Edin, J. Peters, "Grip stabilization of novel objects using slip prediction," IEEE Trans. Haptics, 11(4), 2018.